高级软件工程（研究生）

被检查仓库统一约定与样板仓库规范

目录结构 · 编号规范 · 留痕文件 · 报告 JSON

common 公共脚手架 + good 全绿样板 + bad 六类标准样例验收

一、为什么需要统一契约

三个子项目（A 过程采集、B 需求—测试追溯、C AI 留痕合规）都要自动读取各组的仓库。如果每组目录命名、编号、字段都不一样，系统就无法分析，助教也无法横向比较。因此统一约定一套“被检查仓库规范”。它有双重身份：

- **对 A/B/C 系统：**它是你们所做系统的输入规范——你们要按这个契约去解析目标仓库；
- **对你们自己：**它也是你们自己仓库的交付规范——你们的系统最终要能反过来检查本组仓库。

请严格使用本文件给出的文件路径、编号格式与字段名。凡未按约定导致系统无法解析的，相应客观项判“不通过”；约定之外的创新内容请放在对应目录下、不要改根结构。

二、标准目录结构

```
repo/
├─ team-roster.yml                          # 花名册：学号–姓名–GitLab 账号–角色
├─ assessment.yml                           # 评阅配置：截止讲次、权重、阈值
├─ common/                                  # 模板附带公共脚手架（只读克隆/配置/报告，可直接用）
├─ docs/
│   ├─ m1-立项与需求/
│   │   ├─ NABC.md                          # 立项卡（NABC 四问）
│   │   ├─ SRS.md                           # 需求规格：FR-xx、NFR、权限矩阵
│   │   ├─ RTM.md                           # 需求–测试追溯矩阵（初版，m3 补全）
│   │   └─ 需求审查分析报告.md               # 分析任务产出（随 m1 提交）
│   ├─ m2-架构与设计/
│   │   ├─ architecture.md                  # 构件图、部署、状态机设计
│   │   ├─ ADR.md                           # 架构决策记录（至少 1 条）
│   │   ├─ design.md                        # 类/时序/状态设计（随 L18–L20 补充）
│   │   └─ 架构决策论证报告.md               # 分析任务产出（随 m2 提交）
│   ├─ m3-测试/
│   │   ├─ 测试报告.md                      # 可复现命令、通过率、覆盖率（自证）
│   │   └─ Eval 集.md                       # 标准样例仓库与最小评测集
│   └─ m4-交付/
│       ├─ 用户手册.md
│       ├─ 变更影响分析.md
│       └─ 风险评估与 AI 使用说明.md         # 分析任务产出（随 m4 提交）
├─ src/                                     # 系统源码（可按语言分子目录）
├─ tests/                                   # 测试用例 TC-xx，含 covers: FR-xx
├─ process/
│   ├─ ai-log.csv                           # AI 交互日志（append-only，带哈希链）
│   └─ review-checkpoints.yml               # 人工评审检查点登记
└─ README.md                                # 项目说明、运行方式、目录索引
```

路径 / 文件	用途	何时要有
team-roster.yml	成员与 GitLab 账号映射，是贡献核查的基础	L17 组队后

路径 / 文件	用途	何时要有
assessment.yml	里程碑截止讲次、权重、阈值等评阅配置（不依赖 CI）	L17 组队后
common/	模板附带公共脚手架：只读克隆、配置读取、报告 JSON 基类，可直接用或改写	随模板，可改
docs/m1-立项与需求/	NABC、SRS、RTM 初版；分析任务产出：需求审查分析报告	m1 前（不晚于 L19 课后）
docs/m2-架构与设计/	构件图、ADR、详细设计；分析任务产出：架构决策论证报告	m2 前（不晚于 L20 课后）
src/	系统源码，L20 后最小内核可运行	m2 起持续提交
tests/	结构化测试用例，每条注明 covers	m3 前（L25 前）
docs/m3-测试/	测试报告（自证）与评测集	m3 前（L25 前）
docs/m4-交付/	用户手册、变更影响分析；分析任务产出：风险评估与 AI 使用说明	m4 前（L27 前）
process/ai-log.csv	AI 交互留痕（append-only）	全程随用随记
process/review-checkpoints.yml	各阶段人工评审点	每里程碑

2.1 common/ 公共脚手架（随模板包提供）

common/ 是与业务无关的通用底座，目的是让各组不要重复造轮子、并统一报告格式。它只包含三类能力，核心检查规则仍由各组自行实现：

文件	提供能力	边界（不替你做什么）
common/repo.py	只读克隆封装（带超时、只读令牌）、执行只读 git 命令的统一入口	不实现贡献统计、时间窗判定等 A 规则；不 push、不写仓库
common/config.py	读取 assessment.yml 与 team-roster.yml，按子项目取阈值	不内置任何具体规则阈值，阈值以配置文件为准
common/report.py	报告 JSON 基类（公共字段、问题码容器、UTF-8 落盘）	不替你判定问题；issues 由你自己的规则填入

common/ 仅依赖 Python 标准库与 PyYAML（模板 requirements.txt 已列），不依赖任何 CI Runner；可以直接使用、修改或不用。它不包含“执行被测仓库构建/测试”的能力——本项目不自动执行测试（A 该项为选做）。

三、花名册 team-roster.yml

字段	含义	要求
team.id	组号	如 T07，由助教统一分配
project	所选子项目	取值 A / B / C，三选一
student_id	学号	本人真实学号
name	姓名	本人真实姓名

字段	含义	要求
gitlab	GitLab 账号	本人实名账号，全组唯一，不得共用
role	角色	组长/组员
modules	负责模块	便于核对贡献，与个人贡献陈述一致

```yaml
course: 高级软件工程
term: 2026 秋
team:
  id: T07
  project: A
  members:
    - student_id: 2026001
      name: 张三
      gitlab: zhangsan
      role: 组长
      modules: 采集引擎、时间窗判定
    - student_id: 2026002
      name: 李四
      gitlab: lisi
      role: 组员
      modules: 贡献统计、看板
    - student_id: 2026003
      name: 王五
      gitlab: wangwu
      role: 组员
      modules: 配置读取、报告导出
    - student_id: 2026004
      name: 赵六
      gitlab: zhaoliu
      role: 组员
      modules: 留痕对接、文档
```

A 系统的硬规则：花名册每位成员必须映射到唯一 GitLab 账号；映射不上（未匹配）或无被合并贡献（零贡献）都会报异常。请在 L17 组队后立即建立并合入主干。

四、编号规范

标识	格式	示例	用于
功能需求	FR-两位数字	FR-05	SRS 中每条功能需求；关键需求加 critical 标记
非功能需求	NFR-两位数字	NFR-03	可度量质量需求，建议注明类别与指标
测试用例	TC-两位数字	TC-22	tests/ 下用例，必须含 covers 字段
架构决策	ADR-三位数字	ADR-001	架构决策记录，含背景/候选/决策/后果
里程碑标签	m1-requirements 等（见 4.1）	m3-testing	Git 语义化标签，按顺序、截止前打
评审检查点	CP-mN	CP-m3	每个里程碑一个人工评审点

4.1 里程碑标签的打法（语义化）

里程碑标签统一使用带说明的语义化名称（annotated tag），名称本身即说明阶段含义：

```
git tag -a m1-requirements -m "m1 需求基线"
git tag -a m2-architecture -m "m2 架构与最小内核"
git tag -a m3-testing -m "m3 测试质量"
git tag -a m4-release -m "m4 交付"
git tag -a final -m "终交"
```

里程碑	标签名	含义	截止
m1	m1-requirements	立项与需求基线	L19 前
m2	m2-architecture	架构与最小内核	L20 前
m3	m3-testing	测试质量	L25 前
m4	m4-release	交付	L27 前
final	final	终交	L28 前

系统按 m1/m2/m3/m4 前缀识别里程碑，早期若已打过 m1、m2 这类短标签也兼容；本学期统一使用语义化名称，不要使用 m1-final、final1 等非规范写法。

4.2 用例的 covers 字段（B 系统核心）

每条测试用例必须在文件中明确写出它覆盖哪条需求，B 系统据此建立追溯矩阵。推荐写法（Markdown 或结构化注释均可，字段名 covers 必须一致）：

```
用例编号: TC-18
covers: FR-05
类型: 正常 / 边界 / 无效等价类
前置条件: 已存在组号 T07 的花名册
步骤: 1. 绑定仓库链接  2. 触发一次检查
预期结果: 输出每位成员贡献，并给出阶段 tag 状态
实际结果: PASS / FAIL
```

追溯规则	违反时报错
每条 FR 至少有 1 条 PASS 用例；critical 的 FR 必须同时有正常用例与边界用例	MISSING_TEST（漏测）/ CRITICAL_NO_BOUNDARY / CRITICAL_NO_HAPPY
每条 TC 必须引用一条真实存在的 FR	ORPHAN_TEST（孤儿用例）
编号不得重复、引用不得指向不存在的条目；用例内容不得重复	DUP_ID / BAD_REF / DUP_TEST
用例实际结果为 FAIL 不得计为已覆盖（全 FAIL 却标绿）	TEST_ALL_FAIL_MARKED_COVERED
某 FR 被修改/删除，关联用例自动置“待回归”并给出影响清单	NEED_REGRESSION

五、AI 交互日志 process/ai-log.csv

凡使用 AI 辅助需求、设计、代码、测试，都要逐次登记。CSV 字段固定如下，字段名使用英文、内容用中文填写；单个单元格内不要使用英文逗号（可用顿号或空格）。日志按时间顺序追加、不得删除或改写历史（append-only），并由 C 系统用哈希链校验完整性。

字段	含义	填写要求
date	日期	YYYY-MM-DD
phase	阶段	m1 / m2 / m3 / m4
tool	AI 工具	如实填写所用工具
prompt	给 AI 的输入要点	可概括但要能对应具体问题
artifact	产出物	文件名或文档章节
commit	关联提交	可定位的提交号，无则留空
reviewer	人工审查人	必须是本人或组员姓名，且不得与提交者始终为同一人
ai_decision	AI 结论/初稿	简述 AI 给了什么
human_action	人工动作	接受 / 接管 / 驳回，并写明改了什么；只写“接受”无说明视为未实质审查
owner	决策人	对该产出负最终责任的人

```
date,phase,tool,prompt,artifact,commit,reviewer,ai_decision,human_action,owner
2026-10-29,m2,某 CodingAgent,生成贡献采集模块的提交解析,parse_commits.py,3f9a2c1,张三,AI 初稿,接管并重写时间窗判定逻辑,张三
2026-11-05,m3,某测试 Agent,为贡献统计补边界用例,test_window.py,81bc4d2,李四,AI 初稿,接受并补一条无效等价类,李四
2026-11-08,m3,某对话助手,判断覆盖率阈值是否合理,评审记录,无,王五,建议 0.5,驳回改为 0.6 并说明依据,王五
```

C 系统的硬规则：每条 AI 产出必须有 reviewer 与 human_action；进入主干的 AI 代码必须有关联 commit 且有“接管/审查”记录；缺一项即报“未审查 AI 代码”。human_action 只写“接受”而无任何说明的，视为未实质审查。日志断链、删除或篡改报 LOG_TAMPERED。

六、评审检查点 review-checkpoints.yml

每个里程碑必须有一次人工评审（可线上合并请求评审或线下评审会），记录评审人、结论与证据。

```yaml
checkpoints:
  - id: CP-m1
    milestone: m1
    name: 需求人工评审
    reviewer: 组长、张三
    date: 2026-10-31
    result: 通过
    evidence: docs/m1-立项与需求/评审记录.md
  - id: CP-m3
```

```
    milestone: m3
    name: 测试用例人工评审
    reviewer: 李四
    date: 2026-11-18
    result: 打回
    note: 关键需求缺边界用例，补无效等价类后复审
```

result 取值	含义
通过	该里程碑人工评审通过，可进入下一阶段
打回	存在问题，需重做并复审；复审记录追加保留，缺复审即推进报 CHECKPOINT_NOT_FOLLOWED

七、评阅配置 assessment.yml

评阅配置只放“里程碑截止讲次、评分权重、过程阈值”，不包含构建/测试/覆盖率等需要执行被测仓库的项（本项目不依赖 CI Runner；测试由各组本地自证、现场核验复现）。文件名固定为 assessment.yml，避免与 GitLab 流水线文件 .gitlab-ci.yml 混淆。

```
milestones:
  m1: {name: 需求基线, tag: m1-requirements, due_lecture: L19, required: NABC 与 SRS 与 RTM 初版}
  m2: {name: 架构与最小内核, tag: m2-architecture, due_lecture: L20, required: 构件图与 ADR 与 最小内核可运行}
  m3: {name: 测试质量, tag: m3-testing, due_lecture: L25, required: 测试报告 与 标准样例问题全部报出}
  m4: {name: 交付, tag: m4-release, due_lecture: L27, required: 用户手册 与 变更影响分析}
weights:
  team: 70              # 团队分（全组相同）
  ta_individual: 22     # 助教评价（对个人）
  peer: 8               # 组内互评（对个人，零和，n 人组总盘 8n）
# 个人贡献红线：核验认定无实质贡献、无留痕、讲不清本人部分者，个人分清零且总评封顶
process_thresholds:
  min_merged_contributions: 5
  active_weeks: 3
  bigbang_window_hours: 24
traceability_thresholds:
  min_cases_per_fr: 1
  critical_requires_boundary: true
  critical_requires_happy: true
ai_thresholds:
  checkpoint_required_per_milestone: true
  ai_code_requires_review: true
  log_hash_chain: true
```

配置	含义
tag / name	各里程碑的语义化标签名与中文名（m1-requirements 等）
due_lecture	各里程碑截止讲次（不写具体日期）

配置	含义
weights	项目分构成：团队 70 / 助教评价（个人）22 / 组内互评（个人）8；现场核验不占分、触发式开展，另设个人贡献红线。
process_thresholds	A 用：每人最低被合并贡献数、最少活跃周数、大爆炸窗口（小时）
traceability_thresholds	B 用：每条 FR 至少用例数、关键 FR 是否要求正常/边界用例
ai_thresholds	C 用：每里程碑是否必须有评审点、AI 代码是否必须审查、日志是否启用哈希链

测试通过率、覆盖率、lint 等不进入本配置：它们由各组在 m3 测试报告中自行写明可复现命令与结果，现场核验复现核对；系统不自动执行、也不依赖任何 CI Runner。

八、三个子系统的报告 JSON

A/B/C 统一输出 JSON：包含组号、被检查提交、检查时间、客观项结果、异常清单（含问题码与证据）。下列为示意，键名与枚举值请保持一致，便于集成汇总。

8.1 A 报告（过程采集与贡献核查）

```
{
  "team_id": "T07",
  "project": "A",
  "commit": "a1b2c3d",
  "checked_at": "2026-11-20T10:00:00+08:00",
  "read_only": true,
  "milestones": [
    {"tag": "m1-requirements", "on_time": true},
    {"tag": "m3-testing", "on_time": false, "missing": true}
  ],
  "members": [
    {"student_id": "2026001", "gitlab": "zhangsan", "merged_commits": 42,
"active_weeks": 5, "flag": "OK"},
    {"student_id": "2026004", "gitlab": null, "merged_commits": 0, "active_weeks":
0, "flag": "ZERO_CONTRIB"}
  ],
  "self_attested_test": {
    "source": "docs/m3-测试/测试报告.md",
    "command": "python -m pytest",
    "pass_rate": 0.96,
    "coverage": 0.71,
    "note": "self-attested by students; rechecked at spot audit, not executed by the
system"
  },
  "issues": [
    {"code": "SOLO_COMMIT", "severity": "HIGH", "evidence": "95% 提交来自同一账号"},
    {"code": "BIG_BANG", "severity": "MEDIUM", "evidence": "m4-release 截止前 24 小时
集中提交"}
  ]
}
```

说明：self_attested_test 只搬运 m3 测试报告中的自证数据（命令、通过率、覆盖率），不代表系统已执行或验证；测试结果由现场核验复现核对。

8.2 B 报告（需求—测试追溯一致性）

```
{
  "team_id": "T07",
  "project": "B",
  "commit": "a1b2c3d",
  "read_only": true,
  "requirements": {
    "total": 24, "covered": 22,
    "uncovered": ["FR-05"],
    "critical_missing_boundary": ["FR-02"],
    "critical_missing_happy": []
  },
  "tests": {"total": 61, "orphan": ["TC-22"], "duplicate": [],
"all_fail_marked_covered": []},
  "coverage": {"fr": 0.92, "critical_boundary": 0.88},
  "regression": {
    "changed_fr": ["FR-05"],
    "affected_tests": ["TC-18"],
    "pending": ["TC-18"],
    "status": "NEED_REGRESSION"
  },
  "issues": [
    {"code": "MISSING_TEST", "ref": "FR-05", "line": 42, "severity": "HIGH"},
    {"code": "ORPHAN_TEST", "ref": "TC-22", "line": 210, "severity": "MEDIUM"}
  ]
}
```

8.3 C 报告（AI 协同留痕与评审检查点）

```
{
  "team_id": "T07",
  "project": "C",
  "commit": "a1b2c3d",
  "read_only": true,
  "ai_log": {"entries": 38, "reviewed": 35, "missing_review": 3, "field_missing":
1},
  "ai_log_integrity": {"hash_chain": "BROKEN", "tampered_entries": ["ai-log#31"]},
  "checkpoints": {"required": 4, "passed": 3, "missing": ["m4"]},
  "ai_code_release": {"to_main": 6, "reviewed": 5, "blocked": ["d4e5f60"]},
  "issues": [
    {"code": "AI_CODE_NO_REVIEW", "commit": "d4e5f60", "severity": "HIGH"},
    {"code": "MISSING_CHECKPOINT", "milestone": "m4", "severity": "HIGH"},
    {"code": "AI_FINALIZED_KEY_DECISION", "topic": "选型", "severity": "HIGH"},
    {"code": "LOG_TAMPERED", "entry": "ai-log#31", "severity": "HIGH"}
  ]
}
```

公共字段	含义
team_id / project	组号 / 子项目（A/B/C）
commit	本次检查对应的提交，保证可复现

公共字段	含义
read_only	恒为 true，声明只读
issues[].code	问题码（见第十章）
issues[].severity	HIGH / MEDIUM / LOW
issues[].evidence / line / ref	证据：说明、行号或关联编号，必须可定位
self_attested_test	仅 A 搬运测试报告中的自证数据，不代表系统已验证

九、good 样板仓库（全绿）

good 仓库是“规范交付”的范例，A/B/C 三系统对它检查应无 HIGH 问题、全部为绿。它满足：

- 花名册 4 人全部映射到唯一账号，每人有不少于阈值的被合并贡献、活跃不少于 3 周，且分布在整个周期；
- m1-requirements/m2-architecture/m3-testing/m4-release 与 final 标签齐全、均在截止讲次前、顺序合规，无末日集中；
- 24 条 FR 全部有通过用例，关键 FR 都有正常与边界用例，无孤儿用例、无重复编号；需求变更的用例已回归；
- 每次 AI 交互均有日志且哈希链完整，4 个评审检查点均通过，进入主干的 AI 代码均有审查/接管记录；
- m3 测试报告写明可复现命令、通过率与覆盖率（现场核验时可复现）；关键决策均标“人工决策”。

期望结果：A 报告 issues 为空、贡献与阶段全绿（self_attested_test 仅原样呈现）；B 报告覆盖率 100%、无差异、回归闭环；C 报告无缺失、无断链、无违规。可随骨架包提供的 good 样例用于你们自测解析与判定逻辑。

十、bad 标准样例仓库（六类问题，验收标准）

bad 仓库是 A/B/C 共同的验收测试固件：故意预置下面六类典型问题，系统必须在正确报出。这是项目“硬规则”是否落地的核心证据。

编号	预置的问题	具体做法	应由谁报	期望问题码
R1	一人提交、其余搭便车	4 人花名册中 95% 以上被合并提交来自同一账号，其余 3 人贡献为 0	A	SOLO_COMMIT + ZERO_CONTRIB
R2	缺 m3 阶段	没有 m3-testing 标签，也没有 docs/m3-测试/ 目录	A	MISSING_MILESTONE

编号	预置的问题	具体做法	应由谁报	期望问题码
R3	关键需求漏测	FR-05 存在但无任何用例覆盖；FR-02 为关键需求却无边界用例	B	MISSING_TEST（FR-05）+ CRITICAL_NO_BOUNDARY（FR-02）
R4	孤儿用例	TC-22 的 covers 指向不存在的 FR-99	B	ORPHAN_TEST（TC-22）
R5	AI 代码无审查	提交信息含 [AI-GEN] 的 src/parser.py 提交，在 ai-log 中无对应审查/接管记录（ai-log 未登记该提交）	C	AI_CODE_NO_REVIEW
R6	末日大爆炸提交	约 85% 提交集中在 m4-release 截止前 24 小时，过程中段长期断档	A	BIG_BANG

验收要求：A 必须报出 R1、R2、R6；B 必须报出 R3、R4；C 必须报出 R5。每类问题都要给出可定位证据（编号/提交/行号）。漏报任意一类 HIGH 问题，相应子项目“标准样例验收”项不得分；误报需在报告中可被证伪并说明判定依据。六类问题均不依赖 CI Runner。

附录：客观检查项速查

子项目	客观可判定项（系统只读、可复现）	不做（归人工/自证）
A	连通性、目录齐全、花名册映射、被合并贡献与活跃周数、tag 时间窗、末日集中、README-only/同邮箱等异常、分支保护	不自动执行构建/单测/覆盖率/lint（仅选做）；测试通过率/覆盖率由 m3 自证、现场核验复现；贡献含金量、是否代做归人工
B	FR/TC 编号、covers 引用、覆盖率、孤儿/漏测/重复/全 FAIL 标绿、跨版本变更与待回归（结构化输入）	用例设计好坏、非结构化文本的最终追溯
C	AI 日志字段齐全、评审点存在、AI 代码审查记录、关键决策人工签名、日志哈希链是否断链、审查人是否未交叉	审查质量、AI 生成比例、是否真接管